Übung 3: Asymptotische Analyse rekursiver Algorithmen

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, die Laufzeit rekursiver Algorithmen in Abhängigkeit von der Eingabegröße n zu modellieren, Rekursionsgleichungen aufzustellen und deren asymptotische Laufzeit mithilfe der Θ-Notation und der Mastermethode zu bestimmen.

1. Kreuzen Sie die Aussagen an, die wahr sind.
   A. Im Worst Case hat die Suche nach einem Element in einem sortierten Array mittels binärer Suche die asymptotische Laufzeit $O(n)$.
   B. Im Worst Case hat die Suche nach einem Element in einem sortierten Array mit Hilfe der binären Suche die asymptotische Laufzeit $\Omega(n)$.
   C. Es gibt Funktionen, die sowohl in $\Omega(n)$ als auch in $O(n^2)$ liegen.
   D. Wenn eine Funktion in $\Theta(n)$ liegt, dann liegt sie auch in $O(n^2)$.
   E. Wenn eine Funktion in $\Omega(n)$ liegt, dann liegt sie immer auch in $O(n)$.

2. Bestimmen Sie für jeden der folgenden Codeblöcke die asymptotische Laufzeit und geben Sie sie in der Θ-Notation an.

   (a)
   ```
   int result = 0;
   for (int i = 0; i < n; i++) {
     for (int j = 0; j < i; j++) {
       result++;
     }
   }
   ```

   (b)
   ```
   int result = 0;
   for (int i = n; i >= 1; i /= 2) {
     result += i;
   }
   ```

   (c)
   ```
   public IList<String> repeat(DoubleLinkedList<String> list, int n) {
     IList<String> result = new DoubleLinkedList<String>();
     for(String str : list) {
       for(int i = 0; i < n; i++) {
         result.addLast(str);
       }
     }
     return result;
   }
   ```

   (d) *Hinweis: In dieser Aufgabe bezeichnet* `LinkedList` *eine einfach verkettete Liste und* `DoubleLinkedList` *eine doppelt verkettete Liste.*

   ```
   public IList<String> repeat(DoubleLinkedList<String> list, int n) {
     IList<String> result = new LinkedList<String>();
     for(String str : list) {
       for(int i = 0; i < n; i++) {
         result.addLast(str);
       }
     }
     return result;
   }
   ```

(e)
```
public int num(int n){
  if (n < 10) {
    return n;
  } else if (n < 1000) {
    return num(n - 2);
  } else {
    return num(n / 2);
  }
}
```

3. Betrachten Sie die folgende Methode.

   Stellen Sie zunächst eine Rekursionsgleichung für die Worst-Case-Laufzeit $T(n)$ der Methode `g` auf. Bestimmen Sie anschließend die asymptotische Laufzeit in der Θ-Notation.

```
public static int g(int n) {
 if (n <= 1) {
  return 1000;
 }
 if (g(n / 3) > 5) {
  for (int i = 0; i < n; i++) {
   System.out.println("Hello");
  }
  return 5 * g(n / 3);
 } else {
  for (int i = 0; i < n * n; i++) {
   System.out.println("World");
  }
  return 4 * g(n / 3);
 }
}
```

4. Verwenden Sie für jede der folgenden Rekursionen die Mastermethode, um die asymptotische Laufzeit in der Θ-Notation zu bestimmen. Falls die Mastermethode nicht anwendbar ist, begründen Sie dies.

   (a)
   $$T(n) = \begin{cases} 18 & \text{falls } n \leq 5 \\ 3T\left(\frac{n}{4}\right) + n^2 & \text{sonst} \end{cases}$$

   (b)
   $$T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ 9T\left(\frac{n}{3}\right) + n^2 & \text{sonst} \end{cases}$$

   (c)
   $$T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ \log(n)T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

   (d)
   $$T(n) = \begin{cases} 1 & \text{falls } n \leq 19 \\ 4T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

   (e)
   $$T(n) = \begin{cases} 5 & \text{falls } n \leq 24 \\ 2T\left(n-2\right) + 5n^3 & \text{sonst} \end{cases}$$

5. Bestimmen Sie für jeden der folgenden Codeblöcke die asymptotische Laufzeit und geben Sie sie in der Θ-Notation an.

(a)
```
void f1(int n) {
  int i = 1;
  int j;
  while(i < n*n*n*n) {
    j = n;
    while (j > 1) {
      j -= 1;
    }
    i += n;
  }
}
```

(b)
```
int f2(int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      System.out.println('j = ' + j);
    }
    for (int k = 0; k < i; k++) {
      System.out.println('k = ' + k);
      for (int m = 0; m < 100000; m++) {
        System.out.println('m = ' + m);
      }
    }
  }
}
```

6. Bestimmen Sie die asymptotische Laufzeit der Methode `method1` in der Θ-Notation.

```
public void method1(int n) {
  if (n <= 100) {
    System.out.println(':3');
  } else {
    System.out.println(':D');
    for (int i = 0; i<16; i++) {
      method1(n / 4);
    }
  }
}
```

7. Betrachten Sie die folgende Java-Klasse und die Methode `recursiveSum`.

   Stellen Sie die Rekursionsgleichung für die Laufzeit der Methode `recursiveSum` auf. Verwenden Sie $T(n)$ als Symbol für die Laufzeit bei einer verketteten Liste mit n Elementen. Bestimmen Sie anschließend die asymptotische Laufzeit von $T(n)$ in der O-Notation.

```
public class RecursiveSumLinkedList {

  static class ListNode {
    int val;
    ListNode next;

    ListNode(int val) {
      this.val = val;
    }
  }

  public static int recursiveSum(ListNode head) {
    if (head == null) {
      return 0;
    }
    return head.val + recursiveSum(head.next);
  }

  public static void main(String[] args) {
    // Erstellen einer verketteten Liste: 3 -> 5 -> 2 -> 7 -> 1
    ListNode head = new ListNode(3);
    head.next = new ListNode(5);
    head.next.next = new ListNode(2);
    head.next.next.next = new ListNode(7);
    head.next.next.next.next = new ListNode(1);

    int result = recursiveSum(head);
    System.out.println("Sum: " + result);
  }
}
```

8. Bestimmen Sie die asymptotische Laufzeit der folgenden Rekursion in der Θ-Notation.

$$T(n) = \begin{cases} 100 & \text{falls } n \leq 5 \\ 9T\left(\frac{n}{3}\right) + 3 \cdot n^2 + \log n & \text{sonst} \end{cases}$$

9. Gegeben ist folgende Python-Funktion:

```
def recursive_function(n):
  if n <= 0:
    return 0
  return n + recursive_function(n - 1)
```

   Stellen Sie die Rekursionsgleichung für die Laufzeit der Funktion `recursive_function` auf. Verwenden Sie $T(n)$ als Symbol für die Laufzeit bei einer Eingabe n.

10. Ein Freund schlägt vor, binäre Suche zu verwenden, um die Suche in einer sortierten verketteten Liste zu beschleunigen. Erreicht man damit eine asymptotische Laufzeit von $\Theta(\log n)$? Begründen Sie Ihre Antwort.

11. Geben Sie für die folgende Funktion `sort_me` die Θ-Notation für deren Laufzeit im Worst Case sowie die Θ-Notation für deren Laufzeit im Best Case in Abhängigkeit von n, der Größe des eindimensionalen Eingabe-Arrays `arr`, an. Welche Eigenschaft hat das Eingabe-Array `arr` im Worst Case und welche Eigenschaft im Best Case? Sie müssen nicht begründen wie Sie auf die Θ-Notation kommen.

```
def sort_me(arr):
  n = len(arr)
  for i in range(n):
    swapped = False  # Flag, um Vertauschungen zu verfolgen
    for j in range(0, n-i-1):
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[j]
        swapped = True

    if not swapped:
      break

# Beispielaufruf
if __name__ == '__main__':
  arr = [4, 2, 7, 1, 9, 3]
  sort_me(arr)
  print("Liste:", arr)
```